



**INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO**



Práctica 1 – Modelo Entidad-Relación

El sistema gestor en un contenedor: Docker

Ejercicio 2A y Capturas 2B

Desarrollo

Avalos Hernandez Joshua Jasiel

Bravo Mina Liam Valapai

Antonio Navarro Domínguez

Grupo: 3CV2

Profesor: Gabriel Hurtado Avilés

El Contenedor

Un contenedor es una instancia ejecutable de una imagen que contiene los archivos, dependencias y configuración necesarios para ejecutar una aplicación. Los contenedores funcionan como procesos aislados del sistema anfitrión y de otros contenedores, pero comparten el kernel del sistema operativo.

Una **máquina virtual (VM)**, en cambio, virtualiza una computadora completa y ejecuta su propio sistema operativo, incluyendo su kernel, controladores y aplicaciones. Por esta razón, una máquina virtual requiere más recursos que un contenedor.

En cuanto al arranque, un contenedor inicia únicamente el proceso necesario para la aplicación, mientras que una máquina virtual debe iniciar un sistema operativo completo. Esto hace que los contenedores tengan generalmente un arranque más rápido. En cuanto al tamaño, las imágenes de contenedores son más ligeras porque no necesitan incluir un sistema operativo completo. Finalmente, respecto al aislamiento, ambos proporcionan separación entre aplicaciones, pero una VM ofrece aislamiento a nivel de sistema operativo completo, mientras que los contenedores comparten el kernel del sistema anfitrión.

Imagen: Es una plantilla de solo lectura que contiene las instrucciones, archivos y dependencias necesarios para crear un contenedor. Una imagen puede utilizarse para crear múltiples contenedores.

Contenedor: Es una instancia ejecutable de una imagen. Puede iniciarse, detenerse, modificarse y eliminarse de manera independiente, manteniendo un entorno aislado para la aplicación.

Volumen: Es un mecanismo de almacenamiento administrado por Docker que permite conservar información fuera del ciclo de vida del contenedor. Los datos almacenados en un volumen permanecen, aunque el contenedor sea eliminado.

Puerto publicado: Es un puerto del contenedor que se hace accesible desde el equipo anfitrión mediante una regla de red. Por ejemplo, `-p 5432:5432` permite que las conexiones al puerto 5432 del equipo anfitrión sean dirigidas al puerto 5432 del contenedor.

La importancia del volumen

En el caso de una base de datos, el volumen es indispensable para conservar los datos independientemente del contenedor. Por defecto, los archivos que se crean dentro de un contenedor se almacenan en su capa de escritura. Si el contenedor se elimina, esa capa

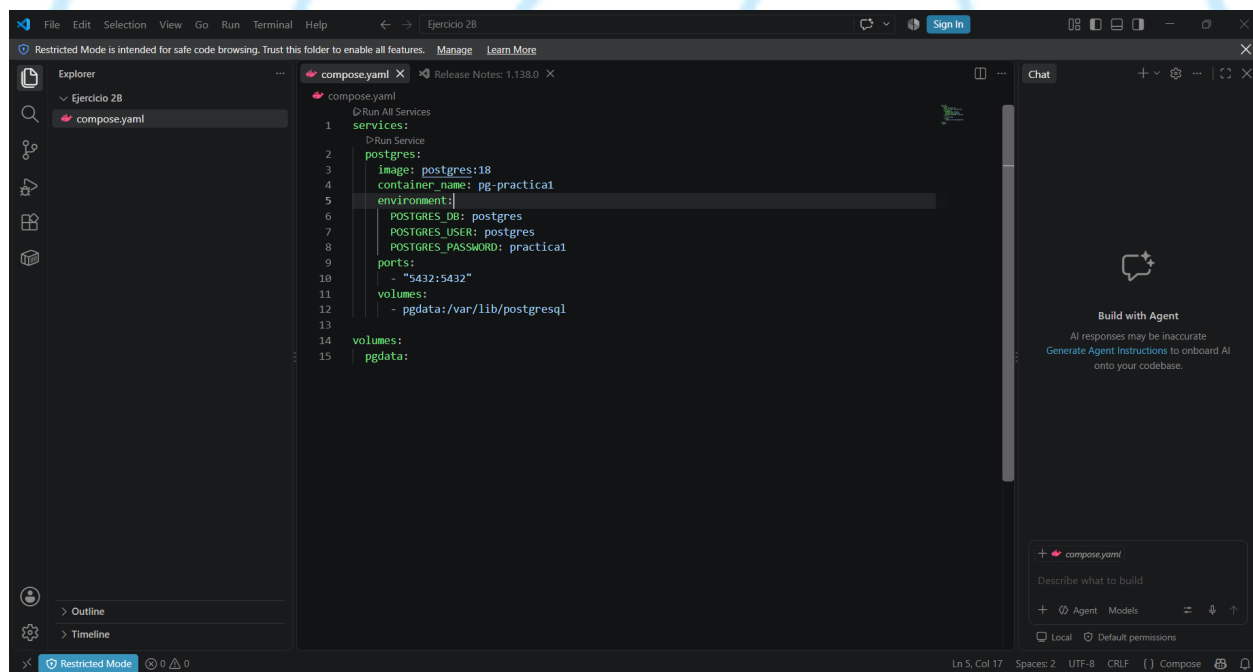
también se elimina y, con ella, los datos que no hayan sido almacenados en un volumen u otro mecanismo de almacenamiento persistente.

Por ejemplo, en PostgreSQL, si se ejecuta un contenedor sin un volumen para almacenar los archivos de la base de datos, la información puede existir mientras ese contenedor exista. Si posteriormente se elimina el contenedor y se crea uno nuevo, la nueva instancia no tendrá los datos anteriores. En cambio, al utilizar un volumen, los archivos de la base de datos permanecen fuera del ciclo de vida del contenedor y pueden ser utilizados por un nuevo contenedor.

Capturas de Ejercicio 2B

Configuración del contenedor mediante Compose

En esta parte se creó el archivo `compose.yaml`, donde se definió la configuración necesaria para ejecutar PostgreSQL 18 mediante Docker. Se especificó la imagen de PostgreSQL, las credenciales de acceso, el puerto de conexión y un volumen llamado `pgdata` para almacenar los datos de forma persistente.



The screenshot shows a code editor with the following content:

```
1 services:
2   postgres:
3     image: postgres:18
4     container_name: pg-practical1
5     environment:
6       POSTGRES_DB: postgres
7       POSTGRES_USER: postgres
8       POSTGRES_PASSWORD: practical1
9     ports:
10      - "5432:5432"
11     volumes:
12      - pgdata:/var/lib/postgresql
13
14 volumes:
15   pgdata:
```

The interface includes a sidebar with 'Ejercicio 2B' and 'compose.yaml', a main editor area, and a right-hand panel with a 'Chat' section and a 'Build with Agent' button.

Conexión a PostgreSQL

En esta etapa se levantó el contenedor de PostgreSQL y se realizó la conexión mediante psql. Posteriormente, se creó la base de datos practica1 y se comprobó la conexión a ella mediante el comando \c practica1, verificando que el gestor estuviera funcionando correctamente.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\joshu\Desktop\Ejercicio 2B> docker compose up -d
[+] up 1/1
    ✔ Container pg-practical Started 0.7s
PS C:\Users\joshu\Desktop\Ejercicio 2B> docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED    STATUS    PORTS                               NAMES
3b897229cb06   postgres:18 "docker-entrypoint.s..." 7 seconds ago Up 7 seconds 0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp pg-practical
PS C:\Users\joshu\Desktop\Ejercicio 2B>
```



Comprobación de persistencia

Finalmente, se detuvo y eliminó el contenedor mediante `docker compose down`, conservando el volumen donde se almacenan los datos. Después, se volvió a levantar el servicio con `docker compose up -d` y se comprobó que la base de datos `practica1` continuaba existiendo. Esto permitió demostrar que los datos permanecen disponibles aunque el contenedor sea eliminado.

```

PS C:\Users\joshu> docker exec -it pg-practical psql -U postgres -d practical
psql (18.6 (Debian 18.6-1.pgdg13+2))
Type "help" for help.

practical=# \l

```

Name	Owner	Encoding	Locale Provider	Collate	Ctype	Access privileges
postgres	postgres	UTF8	libc	en_US.utf8	en_US.utf8	
practical	postgres	UTF8	libc	en_US.utf8	en_US.utf8	
template0	postgres	UTF8	libc	en_US.utf8	en_US.utf8	
template1	postgres	UTF8	libc	en_US.utf8	en_US.utf8	

```

(4 rows)

practical=#

```

```

PS C:\Users\joshu> docker exec -it pg-practical psql -U postgres -d practical
psql (18.6 (Debian 18.6-1.pgdg13+2))
Type "help" for help.

practical=# SELECT current_database();
 current_database
-----
 practical
(1 row)

practical=#

```

Bibliografía

Bibliografía

1. Docker Docs (Documentación oficial - Punto 1 y 2):

Docker Inc. (s. f.). Docker overview. Docker Documentation. Recuperado el 17 de septiembre de 2026, de <https://docs.docker.com/get-started/overview/>

2. Red Hat (Artículo técnico sobre Contenedores vs. VM - Punto 1):

Red Hat. (2026, 24 de junio). Containers vs. VMs: Why not both? Red Hat Topics. <https://www.redhat.com/en/topics/containers/containers-vs-vm>

3. Docker Docs (Manejo de almacenamiento y volúmenes - Punto 2 y 3):

Docker Inc. (s. f.). Manage data in Docker: Use volumes. Docker Documentation. Recuperado el 17 de septiembre de 2026, de <https://docs.docker.com/storage/volumes/>

